

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
Тема: Алгоритм Ахо-Корасика

Студентка гр. 4345

Савонькина Е.Н.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать алгоритм Ахо-Корасик для множественного точного поиска подстрок. Провести исследование его работы на различных размерах входных данных.

Основные теоретические положения.

Задача:

Дан набор строк в алфавите размера k суммарной длины m . Необходимо найти для каждой строки все ее вхождения в текст.

Основная идея алгоритма состоит в построении конечного автомата по набору образцов. Автомат обрабатывает текст посимвольно, переходя из одного состояния в другое. При достижении состояния, соответствующего концу образца, фиксируется вхождение.

Автомат строится в три этапа.

На первом этапе все образцы помещаются в бор, или префиксное дерево. Бор представляет собой дерево, в котором каждое ребро помечено символом, а каждый узел соответствует префиксу одного или нескольких образцов. Общие префиксы образцов хранятся в одном экземпляре, что позволяет сэкономить память. Корень бора соответствует пустой строке. Узлы, в которых заканчивается хотя бы один образец, помечаются как терминальные, и в них сохраняется информация о номерах соответствующих образцов.

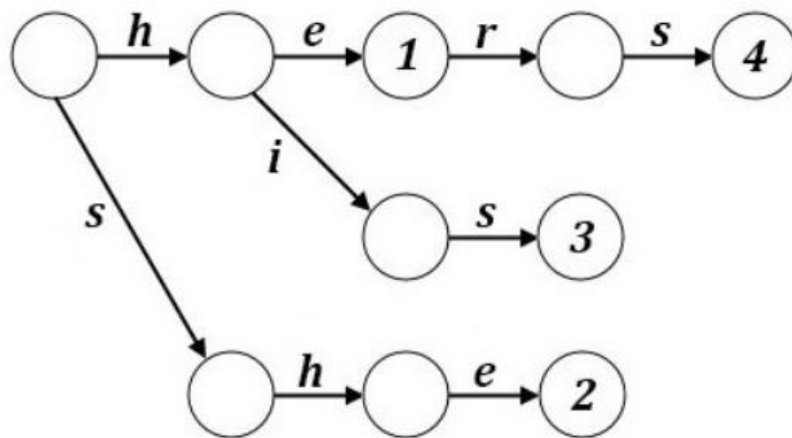


Рисунок 1 – пример построения бора.

На втором этапе для каждого узла бора строится суффиксная (fail-ссылка). Суффиксная ссылка показывает, в какое состояние следует перейти, если дальнейшее продолжение текущего пути по символу текста невозможно. Суффиксные ссылки строятся обходом бора в ширину. Для корня и его непосредственных потомков суффиксная ссылка указывает на корень.

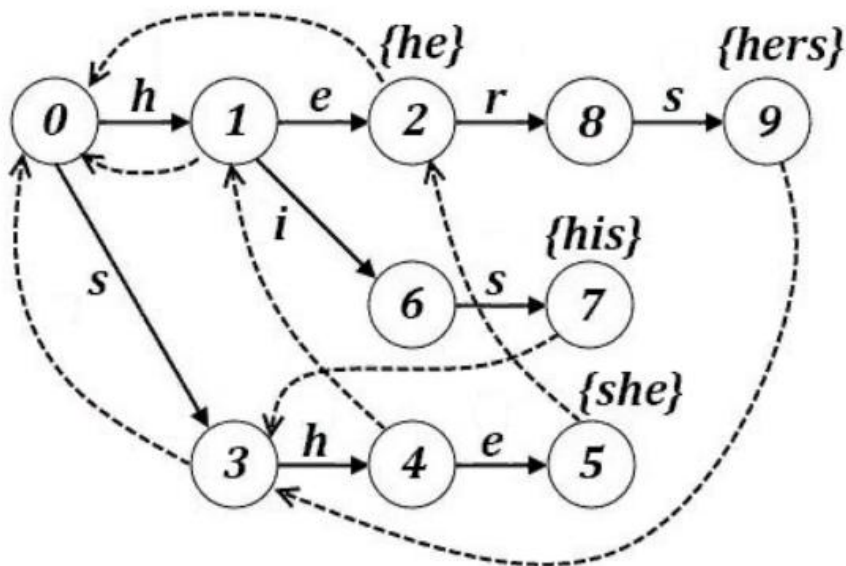


Рисунок 2 – пример бора с суффиксными ссылками.

На третьем этапе для каждого узла строится конечная, или сжатая ссылка, также называемая terminal-ссылкой. Конечная ссылка узла u указывает на ближайший узел в цепочке суффиксных ссылок, в котором действительно заканчивается какой-либо образец. Если в самом узле u есть образец, то конечная ссылка указывает на него самого. Если образца нет, то конечная ссылка наследуется от суффиксной ссылки. Использование конечных ссылок при поиске позволяет быстро собрать все образцы, заканчивающиеся в текущей позиции текста, переходя только по узлам, содержащим образцы, и пропуская промежуточные пустые узлы. Это существенно повышает эффективность поиска.

Обобщением задачи является поиск шаблона с джокерами. Задача состоит в том, чтобы найти все вхождения шаблона, содержащего джокеры, в текст. Алгоритм решения основан на сведении к множественному точному поиску.

Шаблон разбивается на максимальные фрагменты, не содержащие джокеров. Для каждого такого фрагмента запоминается его позиция в исходном шаблоне. Затем все фрагменты ищутся в тексте с помощью алгоритма Ахо-Корасик. Для каждой позиции текста ведётся счётчик количества фрагментов, подтверждающих вхождение полного шаблона с этой позиции. Когда счётчик достигает количества фрагментов, это означает, что все безмасочные части шаблона совпали, а позиции джокеров могут быть заполнены любыми символами текста, и найденное вхождение полного шаблона фиксируется.

Задание 1.

Разработайте программу, решающую задачу точного поиска набора образцов.

Вход:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$).

Вторая - число n ($1 \leq n \leq 3000$, каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выход:

Все вхождения образцов из P в T . Каждое вхождение образца в текст представить в виде двух чисел – i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Выходные данные

NTAG

3

TAGT

TAG

T

Выходные данные:

2 2

Задание 2.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблон образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?ab??c?$ с джокером $??$ встречается дважды в тексте $xabvsscbaababcsaxxabvsscbababcsax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A,C,G,T,N\}$

Вход:

Текст ($T, 1 \leq |T| \leq 1000000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выход:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Входные данные

ACTANCA

A\$\$\$A\$

\$

Выходные данные

1

Вариант 2. Подсчитать количество вершин в автомате; вывести список найденных образцов, имеющих пересечения с другими найденными образцами в строке поиска.

Реализация алгоритма.

Алгоритм Ахо-Корасик реализован в виде класса. Внутри класса хранятся все данные автомата: список переходов для каждого узла, суффиксные ссылки, списки образцов для каждого узла и конечные ссылки.

В начале работы создаётся корневой узел. Каждый узел — это словарь переходов по символам, значение суффиксной ссылки, список номеров образцов и значение конечной ссылки. Словарь переходов позволяет быстро находить следующий узел по символу. Суффиксная ссылка показывает, в какой узел переходить, если текущий путь нельзя продолжить. Конечная ссылка указывает на ближайший узел с образцом по цепочке суффиксных ссылок.

Добавление образца в автомат выполняется посимвольно. Начиная с корня, для каждого символа проверяется, есть ли переход. Если перехода нет, создаётся новый узел. После обработки всей строки в конечном узле сохраняется номер образца.

После добавления всех образцов строится автомат. Обходом в ширину обрабатываются все узлы. Дети корня получают суффиксные ссылки на корень. Для остальных узлов суффиксная ссылка вычисляется через суффиксную ссылку родителя. От родительской суффиксной ссылки ищется переход по тому же символу. Если переход найден, суффиксная ссылка ребёнка указывает на него. Если не найден, поиск продолжается дальше по цепочке суффиксных ссылок, пока не будет найден переход или достигнут корень.

Конечная ссылка вычисляется после суффиксной. Если в узле есть образцы, конечная ссылка указывает на сам этот узел. Если образцов нет, конечная ссылка берётся от узла, на который указывает суффиксная ссылка.

Поиск в тексте происходит за один проход. Для каждого символа текста проверяется, есть ли переход из текущего узла. Если перехода нет, происходит переход по суффиксной ссылке и проверка повторяется. После перехода в новый узел начинается сбор образцов. По цепочке конечных ссылок

извлекаются все номера образцов, заканчивающихся в этой позиции. Для каждого образца вычисляется позиция начала вхождения как текущая позиция в тексте минус длина образца плюс один.

Для поиска шаблона с джокерами алгоритм работает иначе. Сначала шаблон разбивается на части без джокеров. Для каждой такой части запоминается её позиция в исходном шаблоне. Затем все части загружаются в автомат Ахо-Корасик как отдельные образцы.

При поиске в тексте для каждой позиции ведётся счётчик найденных частей. Когда автомат находит очередную часть, вычисляется позиция, с которой должен начинаться полный шаблон. Счётчик для этой позиции увеличивается. Когда счётчик становится равным общему количеству частей, это означает, что все безмасочные части совпали, а позиции джокеров могут быть любыми символами. В этот момент фиксируется вхождение полного шаблона.

После поиска выполняется анализ пересечений образцов. Для каждой пары найденных вхождений проверяется, перекрываются ли их диапазоны в тексте. Два вхождения пересекаются, если начало одного находится внутри другого. Номера всех образцов, участвующих в пересечениях, собираются и выводятся.

Дополнительно реализован подсчёт количества вершин в автомате и поиск пересекающихся образцов. Количество вершин определяется как размер списка children, содержащего все узлы автомата. Каждый узел создаётся при добавлении нового символа образца, если переход по этому символу ранее не существовал. Таким образом, количество вершин равно числу различных префиксов всех образцов плюс корневой узел.

Поиск пересекающихся образцов выполняется после завершения основного поиска. Для каждой пары найденных вхождений проверяется, перекрываются ли их интервалы в тексте. Интервал вхождения определяется от начальной позиции до конечной, которая вычисляется как начальная позиция плюс длина образца минус один. Два вхождения пересекаются, если начало

первого не превышает конец второго и начало второго не превышает конец первого. Образцы, участвующие хотя бы в одном пересечении, собираются в множество. Результат сортируется по номерам образцов и выводится. Если пересечений не найдено, выводится сообщение об отсутствии.

Оценка сложности.

Апостериорная оценка (после работы алгоритма):

Построение бора для всех образцов выполняется за линейное время от их суммарной длины. Каждый символ каждого образца обрабатывается ровно один раз при добавлении в бор. Если суммарная длина всех образцов равна k , то построение бора требует $O(k)$ операций.

Построение суффиксных и конечных ссылок также выполняется за $O(k)$. Обход бора в ширину посещает каждый узел ровно один раз. Для каждого узла поиск суффиксной ссылки может потребовать нескольких переходов по цепочке суффиксных ссылок родителя, однако суммарное число таких переходов по всем узлам ограничено количеством узлов. Это аналогично тому, как работает построение префикс-функции в алгоритме Кнута-Морриса-Пратта, где вложенный цикл не приводит к квадратичной сложности. Таким образом, общее время построения автомата составляет $O(k)$.

Поиск в тексте длины m выполняется за $O(m + z)$, где z — общее количество найденных вхождений. Каждый символ текста обрабатывается за константное время при переходе между состояниями. Хотя при неудачном переходе может потребоваться несколько шагов по суффиксным ссылкам, суммарное число таких шагов за весь поиск не превышает длины текста. Сбор образцов через конечные ссылки требует времени, пропорционального количеству найденных образцов в каждой позиции, что в сумме даёт $O(z)$.

Итоговая временная сложность алгоритма Ахо-Корасик составляет $O(k + m + z)$, где k — суммарная длина образцов, m — длина текста, z — количество вхождений. Это линейное время относительно размера входных данных.

Память требуется $O(k)$ для хранения бора. Каждый узел хранит словарь переходов, суффиксную ссылку, конечную ссылку и список образцов.

Количество узлов не превышает суммарной длины всех образцов плюс один. В худшем случае словарь переходов может содержать до размера алфавита записей на каждый узел, однако на практике используются только существующие переходы.

Для поиска шаблона с джокерами сложность составляет $O(m + p + a)$, где p — длина шаблона, m — длина текста, a — суммарное количество вхождений безмасочных частей в текст. Разбиение шаблона на части выполняется за $O(p)$. Построение автомата для частей требует $O(p)$. Поиск частей требует $O(m + a)$. Обработка счётчиков и проверка условий занимает $O(a)$. Дополнительная память требуется для массива счётчиков размера m .

Таким образом, алгоритм Ахо-Корасик обеспечивает асимптотически оптимальное время решения задачи множественного точного поиска, не зависящее от количества образцов и линейно зависящее только от суммарного размера входных данных и количества найденных вхождений.

Подсчёт количества вершин выполняется за константное время $O(1)$, так как сводится к получению длины списка.

Поиск пересечений в худшем случае требует попарного сравнения всех найденных вхождений. Если общее количество вхождений равно z , то сложность этой операции составляет $O(z^2)$. Проверка пересечения двух вхождений выполняется за $O(1)$, так как требует лишь вычисления границ интервалов и сравнения чисел. В реальных данных количество вхождений обычно невелико, поэтому эта операция не оказывает существенного влияния на общее время работы. При необходимости асимптотически более быстрого поиска пересечений можно использовать заметание или дерево отрезков, что даст сложность $O(z \log z)$.

Априорная оценка (до работы алгоритма):

Оценим сложность до запуска программы в худшем. Если количество образцов равно n , то в худшем случае каждый образец может встречаться практически в каждой позиции текста. Тогда количество найденных вхождений достигает $O(mn)$, а итоговая временная сложность алгоритма принимает вид

$O(k + mn)$, где k – суммарная длина образцов, m – длина текста. Но так как $n \leq k$ получим: $O(k + m + mk) = O(mk)$.

В среднем случае для случайных текстов и образцов, а также для большинства практических задач количество вхождений z пропорционально длине текста: $z = O(m)$. Тогда априорно можно ожидать $O(k + m)$

Память требуется $O(k)$ для хранения бора.

Для алгоритма с джокерами: $O(n + m + k)$, где n – длина текста, m – суммарная длина образцов, k – длина шаблона.

Тестирование.

Таблица тестирования для задачи 1:

Строки	Результат	Комментарии
аывавы 2 ыв ва	Vertex count: 5 2 1 3 2 Peresek: 1 2	Результат корректен.
AAAA 3 А АА ААА	Vertex count: 4 1 1 1 2 1 3 2 1 2 2 3 1 Peresek: 1 2 3	Результат корректен.
GATTACA 2 ATT TAC	Vertex count: 7 2 1 4 2 Peresek: none	Результат корректен.
CCCCC 2 CC CCC	Vertex count: 4 1 1 1 2 2 1 2 2 3 1 3 2 4 1 Peresek: 1 2	Результат корректен.

Таблица тестирования для задачи 2:

Строки	Результат	Комментарии
ACTANCA A\$\$\$ \$	Vertex count: 2 1 Peresek: none	Результат корректен.
xabvccbababcaх ab??с? ?	Vertex count: 7 2 7 Peresek: none	Результат корректен.
AAAA A\$A \$	Vertex count: 2 1 2 Peresek: 1 2	Результат корректен.

Вывод.

В ходе работы был изучен алгоритм Ахо-Корасика и его применение для поиска подстрок. На его основе реализовано эффективное решение задачи поиска шаблона с джокерами, основанный на разбиении шаблона на безмасочные части и применении автомата Ахо-Корасик.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

Задание 1.

```
class AhoCorasick:
    def __init__(self):
        self.children = []
        self.fail = []
        self.output = []
        self.terminal = []
        self.dictionary = []
        self._init_node()

    def _init_node(self):
        self.children.append({})
        self.fail.append(0)
        self.output.append([])
        self.terminal.append(-1)
        node_id = len(self.children) - 1
        print(f"Создан узел {node_id}")
        return node_id

    def add_pattern(self, pattern, pattern_index):
        print(f"\nДобавление образца {pattern_index + 1}: {pattern}")
        node = 0
        for char in pattern:
            print(f" узел {node}, символ {char}")
            if char not in self.children[node]:
                new_node = self._init_node()
                self.children[node][char] = new_node
                print(f" новое ребро: {node} {char} {new_node}")
            else:
                print(f" ребро существует: {node} {char}")
        {self.children[node][char]}
        node = self.children[node][char]
        self.output[node].append(pattern_index)
        print(f" образец {pattern_index + 1} заканчивается в узле {node}")
        print(f" output[{node}] = {self.output[node]}")

    def build(self):
        print("\nПостроение fail и terminal ссылок")
        queue = []
        head = 0
```

```

print("Дети корня получают fail=0")
for char, child in self.children[0].items():
    self.fail[child] = 0
    queue.append(child)
    print(f"    fail[{child}] = 0, в очередь добавлен {child}")

print(f"Очередь: {queue}")

while head < len(queue):
    current = queue[head]
    head += 1
    print(f"\nОбработка узла {current}")

    for char, child in self.children[current].items():
        print(f"    ребро: {current} {char} {child}")
        queue.append(child)
        fail_state = self.fail[current]
        print(f"    начало поиска fail из {fail_state}")

        while fail_state != 0 and char not in self.children[fail_state]:
            print(
                f"        нет {char} в узле {fail_state}, переход по
fail[{fail_state}] = {self.fail[fail_state]}")
            fail_state = self.fail[fail_state]

        if char in self.children[fail_state]:
            self.fail[child] = self.children[fail_state][char]
            print(f"    найден {char} в узле {fail_state}, fail[{child}] =
{self.fail[child]}")
        else:
            self.fail[child] = 0
            print(f"    не найден, fail[{child}] = 0")

        print(f"    fail[{child}] = {self.fail[child]}")

    if self.output[current]:
        self.terminal[current] = current
        print(f"    terminal[{current}] = {current} (есть образцы
{self.output[current]}")
    else:
        self.terminal[current] = self.terminal[self.fail[current]]

```

```

        print(
            f"    terminal[{current}] = terminal[{self.fail[current]}] = {self.terminal[current]} (образцов нет)"

        print("\nПостроенный автомат:")
        for i in range(len(self.children)):
            children_str = " ".join([f"{c}:{v}" for c, v in self.children[i].items()])
            if not children_str:
                children_str = "нет"
            print(
                f"    Узел {i}: ребра={{children_str}}, fail={self.fail[i]}, output={self.output[i]}, terminal={self.terminal[i]}"

        def get_vertex_count(self):
            return len(self.children)

        def search(self, text):
            print(f"\nПоиск в тексте: {text}")
            results = []
            node = 0

            for pos, char in enumerate(text):
                print(f"\n    Позиция {pos}, символ {char}:")
                print(f"        текущий узел: {node}")

                if char not in self.children[node]:
                    print(f"        прямого перехода по {char} из узла {node} нет")

                while node != 0 and char not in self.children[node]:
                    print(f"        переход по fail: {node} {self.fail[node]}")
                    node = self.fail[node]

                if char in self.children[node]:
                    old_node = node
                    node = self.children[node][char]
                    print(f"        переход: {old_node} {char} {node}")
                else:
                    print(f"        остаемся в корне (узел 0)")
                    node = 0

            print(f"        узел после перехода: {node}")

```

```

print(f"    terminal[{node}] = {self.terminal[node]}")

term = self.terminal[node]
if term == -1:
    print(f"    образцов не найдено")

while term != -1:
    print(f"    обработка terminal-узла: {term}")
    for pattern_index in self.output[term]:
        pattern_len = len(self.dictionary[pattern_index])
        start_pos = pos - pattern_len + 1
        print(f"        найден образец {pattern_index + 1}
({self.dictionary[pattern_index]})")
        print(f"        длина={pattern_len}, начало={start_pos + 1}")
        results.append((start_pos + 1, pattern_index + 1))
        term = self.terminal[self.fail[term]]
    if term != -1:
        print(f"    переход к следующему terminal: terminal[fail] =
{term}")

return results

def find_overlapping(self, results):
    print("\nПоиск пересечений:")
    overlapping_patterns = set()

    for i in range(len(results)):
        pos1, pat1 = results[i]
        end1 = pos1 + len(self.dictionary[pat1 - 1]) - 1
        print(f"    вхождение {i}: образец {pat1} ({self.dictionary[pat1 -
1]}), позиции [{pos1}, {end1}]")

        for j in range(i + 1, len(results)):
            pos2, pat2 = results[j]
            end2 = pos2 + len(self.dictionary[pat2 - 1]) - 1
            print(
                f"        проверка с вхождением {j}: образец {pat2}
({self.dictionary[pat2 - 1]}), позиции [{pos2}, {end2}]")

            if pos1 <= end2 and pos2 <= end1:
                print(f"        пересечение найдено: [{pos1}, {end1}] и
[{pos2}, {end2}]")

```

```

        overlapping_patterns.add(pat1)
        overlapping_patterns.add(pat2)
    else:
        print(f"        пересечения нет")

    return sorted(overlapping_patterns)

def main():
    text = input().strip()
    n = int(input())

    ac = AhoCorasick()

    for i in range(n):
        pattern = input().strip()
        ac.dictionary.append(pattern)
        ac.add_pattern(pattern, i)

    ac.build()

    vertex_count = ac.get_vertex_count()
    print(f"\nКоличество вершин: {vertex_count}")

    results = ac.search(text)
    results.sort(key=lambda x: (x[0], x[1]))

    print("\nРезультаты поиска:")
    for pos, pat_num in results:
        print(f"{pos} {pat_num}")

    overlapping = ac.find_overlapping(results)
    print("\nПересекающиеся образцы:")
    if overlapping:
        print("Пересечение:", *overlapping)
    else:
        print("Пересечение: нет")

if __name__ == "__main__":
    main()

```

Задание 2.


```

def find_pattern_with_wildcard(text, pattern, wildcard):
    parts = []
    positions = []
    current_part = []
    current_pos = 0

    print(f"Шаблон: {pattern}, джокер: {wildcard}")

    for i, char in enumerate(pattern):
        if char == wildcard:
            if current_part:
                parts.append(''.join(current_part))
                positions.append(current_pos)
                print(f"Часть {len(parts)}: '{parts[-1]}', позиция в шаблоне {current_pos}")
                current_part = []
                current_pos = i + 1
            else:
                current_part.append(char)

    if current_part:
        parts.append(''.join(current_part))
        positions.append(current_pos)
        print(f"Часть {len(parts)}: '{parts[-1]}', позиция в шаблоне {current_pos}")

    if not parts:
        print("Нет безмасочных частей")
        return [], 0

    print(f"\nБезмасочные части: {parts}")
    print(f"Позиции частей в шаблоне: {positions}")
    print(f"Длина шаблона: {len(pattern)}")

    class AhoCorasick:
        def __init__(self):
            self.children = []
            self.fail = []
            self.output = []
            self.terminal = []
            self._init_node()

```

```

def _init_node(self):
    self.children.append({})
    self.fail.append(0)
    self.output.append([])
    self.terminal.append(-1)
    node_id = len(self.children) - 1
    print(f"    Создан узел {node_id}")
    return node_id

def add_pattern(self, pattern, pattern_index):
    print(f"\n    Добавление части {pattern_index + 1}: '{pattern}'")
    node = 0
    for char in pattern:
        print(f"        узел {node}, символ {char}")
        if char not in self.children[node]:
            new_node = self._init_node()
            self.children[node][char] = new_node
            print(f"        новое ребро: {node} {char} {new_node}")
        else:
            print(f"        ребро существует: {node} {char}")
    {self.children[node][char]}")
    node = self.children[node][char]
    self.output[node].append(pattern_index)
    print(f"        часть {pattern_index + 1} заканчивается в узле {node}")
    print(f"        output[{node}] = {self.output[node]}")

def get_vertex_count(self):
    return len(self.children)

def build(self):
    print("\n    Построение fail и terminal ссылок")
    queue = []
    head = 0

    print("    Дети корня получают fail=0")
    for char, child in self.children[0].items():
        self.fail[child] = 0
        queue.append(child)
        print(f"        fail[{child}] = 0, в очередь добавлен {child}")

    print(f"    Очередь: {queue}")

```

```

while head < len(queue):
    current = queue[head]
    head += 1
    print(f"\n  Обработка узла {current}")

    for char, child in self.children[current].items():
        print(f"      ребро: {current} {char} {child}")
        queue.append(child)
        fail_state = self.fail[current]
        print(f"      начало поиска fail из {fail_state}")

        while fail_state != 0 and char not in
self.children[fail_state]:
            print(
                f"      нет {char} в узле {fail_state}, переход по
fail[{fail_state}] = {self.fail[fail_state]}")
            fail_state = self.fail[fail_state]

        if char in self.children[fail_state]:
            self.fail[child] = self.children[fail_state][char]
            print(f"      найден {char} в узле {fail_state},
fail[{child}] = {self.fail[child]}")
        else:
            self.fail[child] = 0
            print(f"      не найден, fail[{child}] = 0")

        print(f"      fail[{child}] = {self.fail[child]}")

    if self.output[current]:
        self.terminal[current] = current
        print(f"      terminal[{current}] = {current} (есть части
{self.output[current]})")
    else:
        self.terminal[current] = self.terminal[self.fail[current]]
        print(
            f"      terminal[{current}] =
terminal[{self.fail[current]}] = {self.terminal[current]} (частей нет)")

    print("\n Построенный автомат:")
    for i in range(len(self.children)):
        children_str = " ".join([f"{c}:{v}" for c, v in
self.children[i].items()])

```

```

        if not children_str:
            children_str = "нет"
        print(
            f"        Узел {i}: ребра={{{children_str}}},
fail={self.fail[i]}, output={self.output[i]}, terminal={self.terminal[i]}")

def search(self, text):
    print(f"\n Поиск в тексте: '{text}'")
    print(f"    Длина текста: {len(text)}")
    node = 0
    match_counts = [0] * len(text)
    wildcard_matches = []

    for pos, char in enumerate(text):
        print(f"\n        Позиция {pos}, символ {char}:")
        print(f"            текущий узел: {node}")

        if char not in self.children[node]:
            print(f"            прямого перехода по {char} из узла {node}
нет")

        while node != 0 and char not in self.children[node]:
            print(f"            переход по fail: {node} {self.fail[node]}")
            node = self.fail[node]

        if char in self.children[node]:
            old_node = node
            node = self.children[node][char]
            print(f"            переход: {old_node} {char} {node}")
        else:
            print(f"            остаемся в корне (узел 0)")
            node = 0

        print(f"            узел после перехода: {node}")
        print(f"            terminal[{node}] = {self.terminal[node]}")

    term = self.terminal[node]
    if term == -1:
        print(f"            частей не найдено")

    while term != -1:
        print(f"            обработка terminal-узла: {term}")

```

```

        for pattern_index in self.output[term]:
            part_len = len(parts[pattern_index])
            start_pos = pos - part_len - positions[pattern_index] +
1
            print(f"                часть {pattern_index + 1}
('{parts[pattern_index]}') найдена")
            print(f"                часть в шаблоне на позиции
{positions[pattern_index]}")
            print(f"                предполагаемое начало шаблона:
{start_pos}")

            if start_pos >= 0 and start_pos + len(pattern) <=
len(text):
                match_counts[start_pos] += 1
                print(f"                match_counts[{start_pos}] =
{match_counts[start_pos]} (нужно {len(parts)})")

                if match_counts[start_pos] == len(parts):
                    print(f"                ВСЕ ЧАСТИ НАЙДЕНЫ, шаблон
начинается с позиции {start_pos + 1}")
                    wildcard_matches.append((start_pos, start_pos +
len(pattern) - 1))
                else:
                    if start_pos < 0:
                        print(f"                выход за левую границу текста")
                    else:
                        print(
                            f"                выход за правую границу текста
(шаблон длины {len(pattern)} не помещается с позиции {start_pos})")
                        term = self.terminal[self.fail[term]]
                        if term != -1:
                            print(f"                переход к следующему terminal:
terminal[fail] = {term}")

            print(f"\n Итоговый match_counts: {match_counts}")
            return wildcard_matches

ac = AhoCorasick()

for i, part in enumerate(parts):
    ac.add_pattern(part, i)

```

```

ac.build()

vertex_count = ac.get_vertex_count()
wildcard_matches = ac.search(text)

print(f"\nНайденные вхождения шаблона (интервалы в тексте):")
for start, end in wildcard_matches:
    print(f"    [{start}, {end}] (позиция {start + 1})")

return wildcard_matches, vertex_count


def find_overlapping(matches):
    print("\nПоиск пересечений вхождений шаблона:")
    overlapping_matches = set()

    for i in range(len(matches)):
        start1, end1 = matches[i]
        print(f"    вхождение {i + 1}: [{start1}, {end1}]")

        for j in range(i + 1, len(matches)):
            start2, end2 = matches[j]
            print(f"        проверка с вхождением {j + 1}: [{start2}, {end2}]")

            if start1 <= end2 and start2 <= end1:
                print(f"            пересечение: [{start1}, {end1}] и [{start2}, {end2}]")
                overlapping_matches.add(i)
                overlapping_matches.add(j)
            else:
                print(f"            пересечения нет")

    return overlapping_matches


def main():
    text = input().strip()
    pattern = input().strip()
    wildcard = input().strip()

    wildcard_matches, vertex_count = find_pattern_with_wildcard(text, pattern,
wildcard)

```

```

print(f"\nКоличество вершин в автомате: {vertex_count}")

print("\nПозиции вхождений шаблона:")
for start, end in wildcard_matches:
    print(start + 1)

overlapping = find_overlapping(wildcard_matches)
print("\nПересекающиеся вхождения:")
if overlapping:
    print("Пересечение:", *[i + 1 for i in sorted(overlapping)])
else:
    print("Пересечение: нет")

if __name__ == "__main__":
    main()

```